

Reviewer Pack — Minimal Rank of Quantum Groups over Monomial Ideals vs Circu...

Entry 439459b0663d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 22:46:01 UTC

Minimal Rank of Quantum Groups over Monomial Ideals vs Circuit Minimization Problem Complexity

Entry ID: 439459b0663d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 22:46:01 UTC

1. Conjecture

Field A (mathematical branch): Quantum Groups **Field B** (complexity object): Complexity Theory: Circuit Minimization Problem

Statement:

{'text': 'For a given monomial ideal I in the polynomial ring $R[x_1, \dots, x_n]$, define the quantum group G_I as the quotient of the universal enveloping algebra $U(R[x])$ by the ideal generated by elements corresponding to monomials in I . The minimal rank $\rho(G_I)$ is less than or equal to the complexity $C(n)$ of the circuit minimization problem for an n -bit boolean function, where $C(n) = \Theta(2^{n/n})$.' , 'quantitative': ' $E[\rho(G_I)] \leq \Theta(2^{n/n})$ ' , 'counterexample': 'Existence of a monomial ideal I such that $\rho(G_I) > 2^{n/n}$.' }

Rationale (proposer's reasoning):

{'text': 'Quantum groups offer a non-commutative algebraic structure that can encode combinatorial information. Their minimal rank may reflect the complexity of circuits minimizing boolean functions, providing an alternative approach to circuit lower bounds.' , 'explanation': 'The algebraic nature of quantum groups could be leveraged to study complexity theory by examining the ranks of their associated algebras.' }

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bf63c3c88eea4958

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the random n -bit boolean functions ($n \leq 40$), the computed minimal rank $\rho(G_I)$ falls within the range $[C(n)/2, C(n)]$, where $C(n) = \Theta(2^{n/n})$. The conjecture is falsified if any seed results in $\rho(G_I) > C(n)$ or $\text{support_fraction} < 0.8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank quantum group" AND "monomial ideal" AND complexity theory" - "quantum group G_I" AND circuit minimization problem" AND minimal rank" - "polynomial ring U(R[x])" AND monomial ideal" AND circuit complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    C_n = 2**n / n

    # Generate a random n-bit boolean function
    boolean_function = [random.choice([0, 1]) for _ in range(2**n)]

    # Construct a monomial ideal that encodes the function's structure
    # This is a placeholder implementation; actual construction depends on the specific quantum group
    # For simplicity, we assume a trivial encoding where each bit corresponds to a different monomial
    I = set()
    for i in range(2**n):
        if boolean_function[i] == 1:
            monomial = tuple((i >> j) & 1 for j in range(n))
            I.add(monomial)

    # Compute the minimal rank p(G_I) for the quantum group G_I associated with each monomial ideal
    # This is a placeholder implementation; actual computation depends on the specific quantum group
    # For simplicity, we assume a trivial rank calculation where the rank is equal to the number of monomials
    p_G_I = len(I)

    # Measure the complexity C(n) of circuit minimization for each boolean function
    # This is a placeholder implementation; actual computation depends on the specific circuit minimization
    # For simplicity, we assume a trivial complexity calculation where the complexity is equal to the number of monomials
    C_n_measured = n
```

```

return {
    "metric_name": "minimal_rank",
    "metric_value": p_G_I,
    "instances_tested": 1,
    "conjecture_holds": p_G_I <= C_n_measured,
    "counterexample": f"Existence of a monomial ideal I such that  $p(G_I) > \{C_n\_measured\}$ ." if
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Existence of a monomial ideal I such that  $p(G_I) > \{2^{**n}\}$ \"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_rank', 'metric_value': 527, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Ex
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 505, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 517, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 506, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 455, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 500, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 501, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 502, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 524, 'instances_tested': 1, 'conjecture_holds

```

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30e889bb.py", line 65, in <module>

print(f"RESULT: FALSIFIED counterexample=\"Existence of a monomial ideal I such that $p(G_I) > \{2^{**n}\}$ \"")

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that there exists a monomial ideal I such that $\rho(G_I) > 102.4$, which is greater than the complexity $C(n)$ for an n -bit boolea | next: Investigate further into the nature of the monomial ideals that lead to $\rho(G_I)$ exceeding the complexity threshold and explore potential reasons for this deviation.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12674
2	propose	ollama_remote	glm4:latest	0	0	11684
3	preregistration	ollama_remote	glm4:latest	0	0	5985
4	novelty	ollama_remote	glm4:latest	0	0	4747
5	novelty	ollama_remote	glm4:latest	0	0	5581
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15077
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8526
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7273
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9215
10	judge	ollama_remote	glm4:latest	0	0	12144

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 92906 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/439459b0663d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/439459b0663d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/439459b0663d.tar.gz` (if generated)